PROJECT 04

MARGINAL BLOG APPLICATION CI/CD & AMAZON EKS DEPLOYMENT

GitHub • Jenkins • React/Vite • Node.js/Express • SonarQube • Trivy • ECR • EKS • Helm • Prometheus • Grafana

Project detail	Recorded value
Application	Marginal Blog App — React + Vite frontend, Node.js + Express backend, MySQL
Repository	https://github.com/rokit-cloud/Project04-Three-Tier-Blog
AWS region	Asia Pacific (Mumbai) — ap-south-1
Jenkins job	Project04-Blog-CI-CD
EKS cluster	project04-eks — deployed and later deleted during cost cleanup
ECR repositories	blogreact-frontend and blogreact-backend — deleted during cleanup
Report purpose	Implementation steps, exact recorded outputs, correctly placed screenshots, verification and limitations

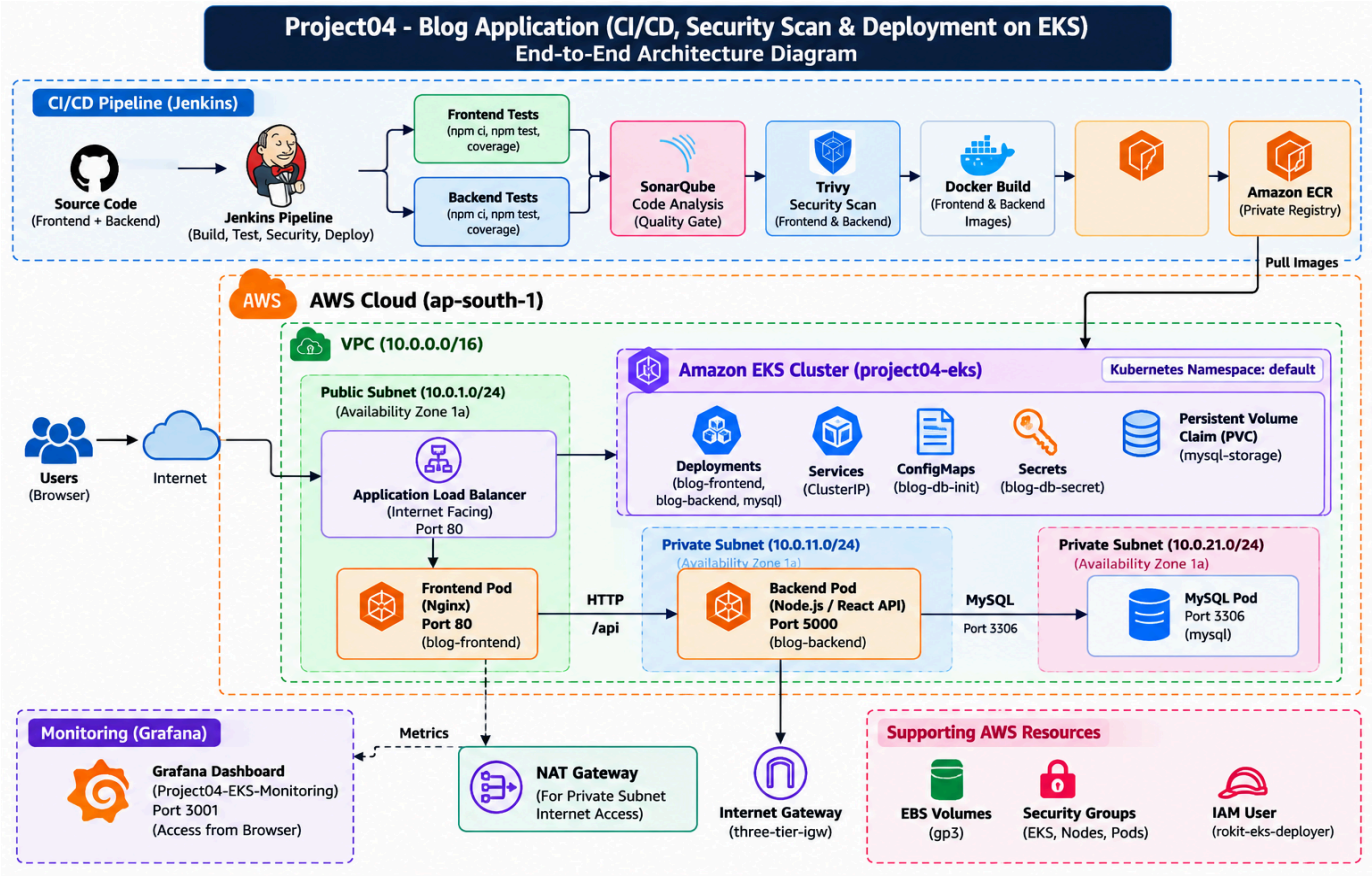
1. Project overview and architecture

Project-04 implements the Marginal blog application as a three-tier workload. The frontend serves the browser interface, the backend exposes API endpoints, and MySQL stores application data. Jenkins automates source checkout, testing, SonarQube analysis, Docker image creation, Trivy scans, ECR publishing and EKS rollout. Prometheus and Grafana provide monitoring.

```
GitHub main branch
      |
      v
Jenkins: Project04-Blog-CI-CD
  |-- Checkout and validate project files
  |-- Backend tests + coverage
  |-- Frontend tests + coverage
  |-- SonarQube analysis / Quality Gate
  |-- Build frontend + backend Docker images
  |-- Trivy image scans
  |-- ECR authentication + image push
  |-- Update Kubernetes deployments
  `-- Verify deployment rollouts
      |
      v
Amazon EKS: project04-eks
  |-- blog-frontend Deployment + Service
  |-- blog-backend Deployment + Service
  |-- mysql StatefulSet (mysql-0) + 5Gi PVC
  `-- monitoring namespace: Prometheus + Grafana
```

Layer	Technology / resource	Role
Presentation	React + Vite; blog-frontend	Marginal browser UI.
Application	Node.js + Express; blog-backend	REST API and health endpoints.
Data	MySQL StatefulSet mysql-0; 5Gi PVC	Relational storage.
CI/CD	Jenkins Project04-Blog-CI-CD	Build, test, analyse, scan and deploy.
Registry	Amazon ECR	Stores versioned frontend/backend images.
Orchestration	Amazon EKS project04-eks	Runs application and monitoring pods.
Observability	Prometheus + Grafana	Metrics, pod health and resource panels.

The frontend screenshot below is the actual browser view captured through localhost port-forwarding. It confirms the UI rendered during the test session; it does not prove a permanent public endpoint.



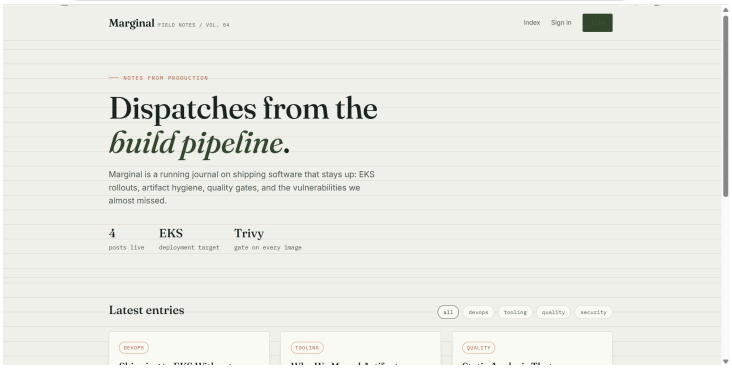


Figure 1. Marginal frontend landing page opened at local forwarded address 127.0.0.1:28082.

2. Source repository and build preparation

GitHub repository: https://github.com/rokit-cloud/Project04-Three-Tier-Blog

```
Project04-Three-Tier-Blog/
|-- frontend/                 React + Vite application
|-- backend/                  Node.js + Express API
|-- database/                 MySQL initialization assets
|-- k8s/                      Kubernetes manifests
|-- Jenkinsfile.ci            Jenkins pipeline definition
|-- sonar-project.properties  SonarQube project configuration
`-- README.md                 Project instructions
```

Step 2.1 — Confirm AWS identity

Where: PowerShell / AWS CloudShell

```
aws sts get-caller-identity
aws configure get region
```

Expected result: Returns the active AWS Account, ARN and UserId. The deployment region used in this project was ap-south-1.

Note: Confirm the account and region before running any create/delete command.

Step 2.2 — Check required tools

Where: PowerShell / Linux shell

```
aws --version
git --version
docker --version
kubectl version --client
helm version
node --version
npm --version
```

Expected result: Each installed CLI prints a version. Install/configure a missing tool before deployment.

Step 2.3 — Clone the repository

Where: PowerShell example

```
New-Item -ItemType Directory -Force D:\DevOps\Project04
Set-Location D:\DevOps\Project04
git clone https://github.com/rokit-cloud/Project04-Three-Tier-Blog.git
Set-Location .\Project04-Three-Tier-Blog
git status
```

Expected result: Git downloads the repository. The recorded Jenkins workspace was /var/lib/jenkins/workspace/Project04-Blog-CI-CD.

Step 2.4 — Validate repository files

Where: Jenkins workspace / repository root

```
test -f docker-compose.yml
test -d frontend
test -d backend
test -f sonar-project.properties
test -f frontend/Dockerfile
test -f backend/Dockerfile
test -f backend/package.json
echo "Project files verified."
```

Expected result: Jenkins log printed: Project files verified.

The source log shows that the pipeline checked for frontend, backend, Dockerfiles, docker-compose.yml and SonarQube configuration before starting the build.

3. Jenkins CI/CD pipeline — correct stage order

The Jenkins job is Project04-Blog-CI-CD. Two separate successful pipeline views were retained: build #21 shows the CI stages through tests, SonarQube, Docker builds and Trivy scans; build #23 shows the expanded CD path including ECR authentication, image push and EKS deployment.

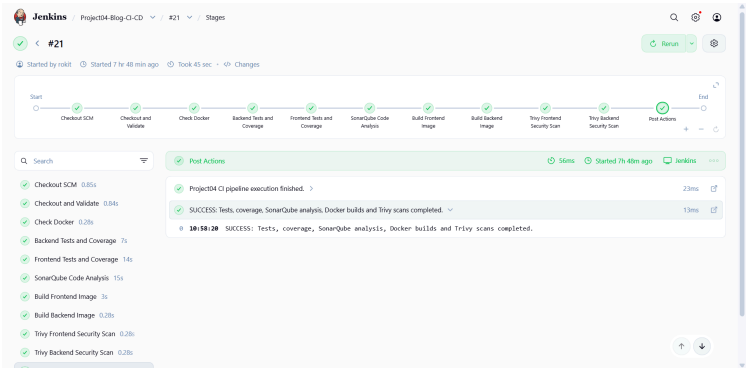


Figure 2. Jenkins build #21 — CI pipeline stages. Each listed stage is green; the final message confirms tests, coverage, SonarQube, Docker builds and Trivy scans completed.

Jenkins stage	What the stage actually does	Recorded result
Checkout SCM	Checks out source from GitHub main.	Completed.
Checkout and Validate	Checks required project files.	Project files verified.
Check Docker	Runs docker --version and docker info.	Docker available.
Backend Tests and Coverage	Runs backend npm ci and Jest tests with coverage.	5/5 tests passed.
Frontend Tests and Coverage	Runs frontend npm ci and Vitest tests.	25/25 tests passed in 5 files.
SonarQube Code Analysis	Analyses frontend/backend project files and LCOV reports.	Analysis successful; gate passed.
Build Frontend Image	Builds blogreact-frontend.	Build completed.
Build Backend Image	Builds blogreact-backend.	Build completed.
Trivy Frontend Security Scan	Scans frontend image for HIGH/CRITICAL vulnerabilities.	0 vulnerabilities in captured build-23 summary.
Trivy Backend Security Scan	Scans backend image for HIGH/CRITICAL vulnerabilities.	0 vulnerabilities in captured build-23 summary.
ECR Authentication / Push	Authenticates Docker and pushes both image tags.	Pushes recorded.
Deploy / Verify EKS	Updates deployment images and waits for rollouts.	Both rollout commands succeeded.

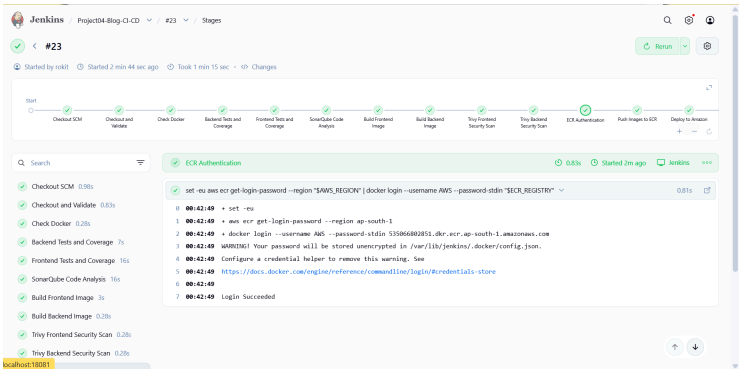


Figure 3. Jenkins build #23 — CD pipeline. This screenshot is the ECR authentication stage within the full green CI/CD stage flow; it is not the SonarQube dashboard.

Recorded ECR login result in build #23: Login Succeeded. Jenkins then pushed the frontend and backend images and updated the Kubernetes deployments.

```
docker push 535066802851.dkr.ecr.ap-south-1.amazonaws.com/blogreact-frontend:build-23
docker push 535066802851.dkr.ecr.ap-south-1.amazonaws.com/blogreact-frontend:ci
docker push 535066802851.dkr.ecr.ap-south-1.amazonaws.com/blogreact-backend:build-23
docker push 535066802851.dkr.ecr.ap-south-1.amazonaws.com/blogreact-backend:ci
echo "Frontend and backend images pushed to ECR."
```

Recorded output: Frontend and backend images pushed to ECR.

4. Frontend and backend tests — separate evidence

4.1 Backend test stage

Jenkins ran the backend test command inside a Node 22 Alpine container. The test runner was Jest.

```
cd backend
npm ci
npm test -- --coverage
```

Captured Jenkins output:

```
PASS tests/server.test.js
  Blog API health and fallback routes
    ✓ GET /healthz returns status ok
    ✓ GET /readyz returns ready when database connection works
    ✓ GET /readyz returns 503 when database connection fails
    ✓ GET an unknown route returns 404
    ✓ GET /api/posts returns posts from the database

Test Suites: 1 passed, 1 total
Tests:       5 passed, 5 total
```

Backend coverage in that Jenkins run: 46.91% statements, 25.00% branches, 37.50% functions and 48.68% lines. These are backend Jest coverage values; do not confuse them with SonarQube New Code coverage.

4.2 Frontend test stage

Jenkins ran the frontend test command in a Node 22 Alpine container. The test runner was Vitest.

```
cd frontend
npm ci
npm test
```

Captured Jenkins output:

```
✓ src/pages/NewPost.test.jsx (5 tests)
✓ src/pages/PostDetail.test.jsx (7 tests)
✓ src/pages/Register.test.jsx (4 tests)
✓ src/pages/Login.test.jsx (4 tests)
✓ src/pages/Home.test.jsx (5 tests)

Test Files  5 passed (5)
Tests       25 passed (25)
```

Frontend coverage in that run: 65.74% statements, 89.28% branches, 73.33% functions and 65.74% lines.

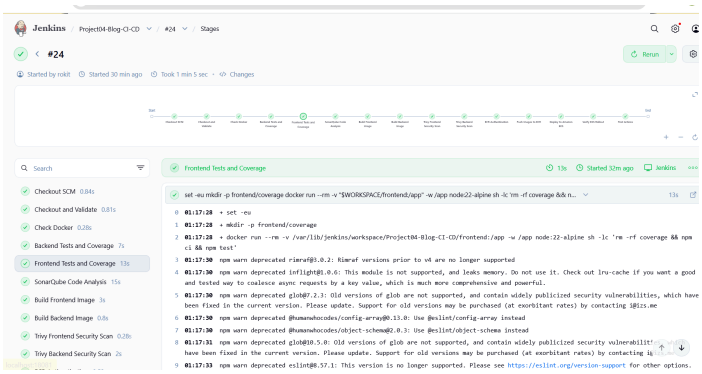


Figure 4. Jenkins frontend test console output showing the frontend test stage. Backend test details above are taken from the retained Jenkins console log; no separate backend-test screenshot is claimed.

Important distinction: Frontend and backend tests are separate Jenkins stages. The screenshot above is the frontend test output; the Jenkins pipeline screenshot in Section 3 shows both test stages in the successful pipeline.

5. SonarQube analysis and Quality Gate — screenshot placement corrected

SonarQube is one combined project analysis for Project04-Three-Tier-Blog. Jenkins supplies both backend and frontend LCOV coverage reports to the scanner. The scanner log explicitly confirms that 4 languages were detected in 26 preprocessed files: CSS, Docker, JavaScript and JSON.

Detected language/profile	Meaning in this project
CSS — Sonar way core	Frontend styling files.
Docker — Sonar way core	Dockerfile analysis.
JavaScript — Sonar way core	React/Vite frontend and Node.js backend source.
JSON — Sonar way comprehensive	JSON configuration/package files indexed by SonarQube; not a separate runtime service.

The scanner command recorded in Jenkins:

```
sonar-scanner \
  -Dsonar.projectKey=Project04-Three-Tier-Blog \
  -Dsonar.projectName=Project04-Three-Tier-Blog \
  -Dsonar.javascript.lcov.reportPaths=backend/coverage/lcov.info,frontend/coverage/lcov.info
```

Jenkins console output: ANALYSIS SUCCESSFUL; SonarScanner Engine completed successfully; EXECUTION SUCCESS. The SonarQube dashboard screenshot is the actual analysis result. The separate Jenkins screenshot is the Jenkins SonarQube Quality Gate integration view.

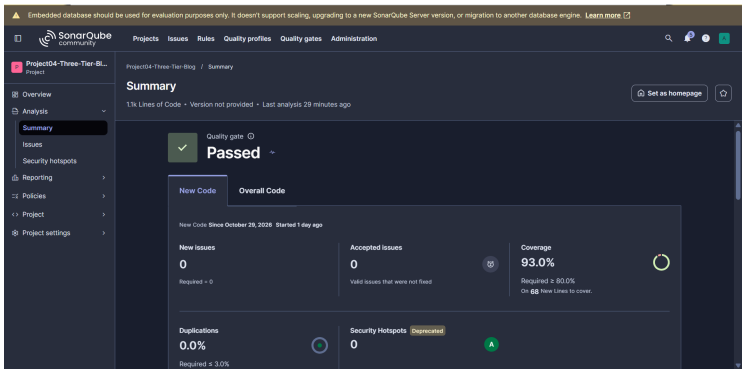


Figure 5. SonarQube project dashboard — Quality Gate Passed. The visible 93.0% is New Code coverage (68 new lines), not the combined overall application test coverage.

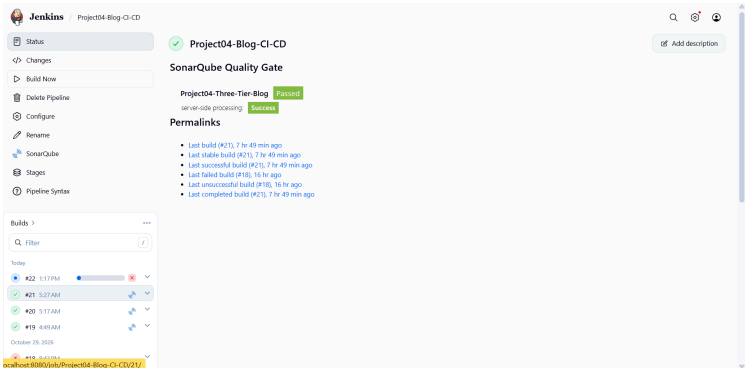


Figure 6. Jenkins SonarQube Quality Gate integration — Project04-Three-Tier-Blog Passed; server-side processing Success. This screenshot belongs in this SonarQube section, not as a generic pipeline screenshot.

Other visible SonarQube New Code values: 0 new issues, 0 accepted issues and 0.0% duplications. The dashboard shows New Code selected. Do not report 93.0% as the overall frontend + backend test coverage.

6. Docker image build and Trivy security scans

6.1 Dockerfiles and image build

Two different Dockerfiles are used: frontend/Dockerfile for the React/Vite UI and backend/Dockerfile for the Express API. The Jenkins build log used Node 22 Alpine and created the following image tags.

```
docker build -t blogreact-frontend:ci -t blogreact-frontend:build-23 frontend
docker build -t blogreact-backend:ci -t blogreact-backend:build-23 backend
```

```
sh-5.2$ cd /var/lib/jenkins/workspace/Project04-Blog-CI-CD && find frontend backend -name Dockerfile -o -name .dockerignore
frontend/.dockerignore
frontend/Dockerfile
backend/.dockerignore
backend/Dockerfile
sh-5.2$ cd /var/lib/jenkins/workspace/Project04-Blog-CI-CD && cat frontend/Dockerfile
FROM node:22-alpine AS build
WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .
ENV VITE_API_URL=/api
RUN npm run build

FROM nginxinc/nginx-unprivileged:stable-alpine AS production

USER root
RUN apk update && apk upgrade --no-cache
USER 101

COPY --from=build /app/dist /usr/share/nginx/html
COPY default.conf.template /etc/nginx/templates/default.conf.template

ENV BACKEND_HOST=backend
ENV BACKEND_PORT=5000

EXPOSE 8080
sh-5.2$
```

Figure 7. Frontend Dockerfile evidence.

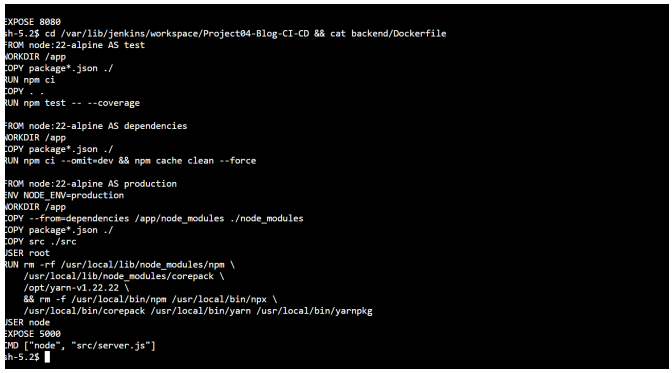


Figure 8. Backend Dockerfile evidence.

The frontend and backend images are separate artifacts. The backend image uses the backend application source; the frontend image packages the browser UI.

6.2 Trivy frontend scan

```
trivy image --severity HIGH,CRITICAL --exit-code 1 blogreact-frontend:build-23
```

Captured build-23 Trivy summary: frontend image based on Alpine 3.24.2; vulnerability count 0 for the displayed Alpine target. The report's `Secrets` column is `-` (not a zero finding count).

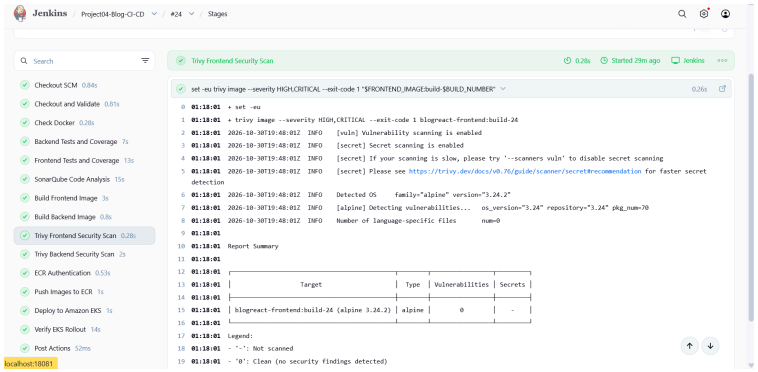


Figure 9. Jenkins build #24 — Trivy Frontend Security Scan. Report Summary shows 0 vulnerabilities for blogreact-frontend:build-24.

6.3 Trivy backend scan

```
trivy image --severity HIGH,CRITICAL --exit-code 1 blogreact-backend:build-23
```

Captured build-23 Trivy summary: backend image based on Alpine 3.24.3 and Node package targets; the displayed targets report 0 vulnerabilities. The retained output shows 0 in the vulnerability column for the listed Alpine and node-pkg targets.

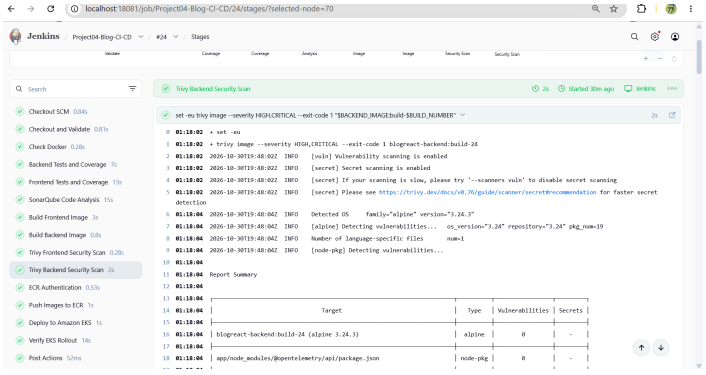


Figure 10. Jenkins build #24 — Trivy Backend Security Scan. Report Summary shows 0 vulnerabilities for the displayed Alpine and Node package targets.

Interpretation: the captured build-23 scans returned zero vulnerabilities for the displayed targets under the HIGH/CRITICAL policy. This does not prove that future image versions remain vulnerability-free; rescan every new build.

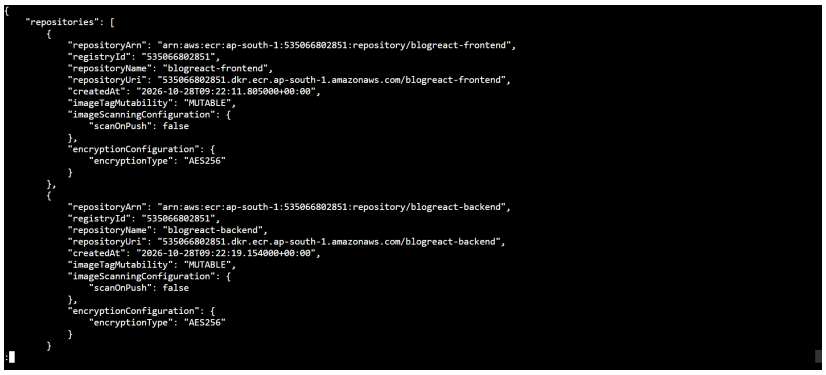
7. Amazon ECR repositories and image publishing

The two ECR repositories are blogreact-frontend and blogreact-backend in account 535066802851, region ap-south-1.

```
aws ecr get-login-password --region ap-south-1 |
  docker login --username AWS --password-stdin 535066802851.dkr.ecr.ap-south-1.amazonaws.com

docker push 535066802851.dkr.ecr.ap-south-1.amazonaws.com/blogreact-frontend:build-23
docker push 535066802851.dkr.ecr.ap-south-1.amazonaws.com/blogreact-frontend:ci
docker push 535066802851.dkr.ecr.ap-south-1.amazonaws.com/blogreact-backend:build-23
docker push 535066802851.dkr.ecr.ap-south-1.amazonaws.com/blogreact-backend:ci
```

Jenkins build #23 recorded `Login Succeeded` and `Frontend and backend images pushed to ECR.`



```
{
  "repositories": [
    {
      "repositoryArn": "arn:aws:ecr:ap-south-1:535066802851:repository/blogreact-frontend",
      "registryId": "535066802851",
      "repositoryName": "blogreact-frontend",
      "repositoryUri": "535066802851.dkr.ecr.ap-south-1.amazonaws.com/blogreact-frontend",
      "createdAt": "2026-10-28T09:22:11.805000+00:00",
      "imageTagMutability": "MUTABLE",
      "imageScanningConfiguration": {
        "scanOnPush": false
      },
      "encryptionConfiguration": {
        "encryptionType": "AES256"
      }
    },
    {
      "repositoryArn": "arn:aws:ecr:ap-south-1:535066802851:repository/blogreact-backend",
      "registryId": "535066802851",
      "repositoryName": "blogreact-backend",
      "repositoryUri": "535066802851.dkr.ecr.ap-south-1.amazonaws.com/blogreact-backend",
      "createdAt": "2026-10-28T09:22:19.154000+00:00",
      "imageTagMutability": "MUTABLE",
      "imageScanningConfiguration": {
        "scanOnPush": false
      },
      "encryptionConfiguration": {
        "encryptionType": "AES256"
      }
    }
```


Figure 11. AWS CLI ECR repository listing showing the frontend and backend repositories.



```
    {
        "registryId": "535066802851",
        "repositoryName": "blogreact-backend",
        "imageDigest": "sha256:1b0e18e6ebcb5859620842478c1da7803f485b7e6d184229df34c04fca988924",
        "imageSizeInBytes": 1561,
        "imagePushedAt": "2026-10-28T09:25:38.737000+00:00",
        "imageManifestMediaType": "application/vnd.oci.image.manifest.v1+json",
        "artifactMediaType": "application/vnd.docker.attestation.manifest.v1+json",
        "subjectManifestDigest": "sha256:e11348d53c262bf5edd05564002b526ca3327975219908cb8a067c48a038e33f",
        "imageStatus": "ACTIVE"
    },
    {
        "registryId": "535066802851",
        "repositoryName": "blogreact-backend",
        "imageDigest": "sha256:e11348d53c262bf5edd05564002b526ca3327975219908cb8a067c48a038e33f",
        "imageSizeInBytes": 65220931,
        "imagePushedAt": "2026-10-28T09:25:38.702000+00:00",
        "imageManifestMediaType": "application/vnd.oci.image.manifest.v1+json",
        "artifactMediaType": "application/vnd.oci.image.config.v1+json",
        "lastRecordedPullTime": "2026-10-30T12:15:47.560000+00:00",
        "imageStatus": "ACTIVE"
    },
    {
        "registryId": "535066802851",
        "repositoryName": "blogreact-backend",
        "imageDigest": "sha256:9ea92a2bc1b10fc643afb969f2d4af05968e1e51239fe27b9113f68d045b1cc7",
        "imageTags": [
            "ci"
        ],
        "imageSizeInBytes": 65220931,
```


Figure 12. ECR image metadata / image tags evidence.

The repositories were later deleted as part of AWS cost cleanup. The screenshots and logs are historical deployment evidence.

8. Amazon EKS cluster and worker node

The historical EKS cluster was project04-eks in ap-south-1. The AWS console screenshot shows the cluster Active with Kubernetes version 1.35.

Step 8.1 — Configure kubectl

Where: AWS CLI host with EKS access

```
aws eks update-kubeconfig --region ap-south-1 --name project04-eks
```

Expected result: AWS CLI adds/updates the project04-eks context in kubeconfig. This only works while the cluster exists.

Step 8.2 — Verify worker nodes

Where: Host with kubectl configured

```
kubectl get nodes -o wide
```

Expected result: The retained output shows the EKS worker node in Ready state.

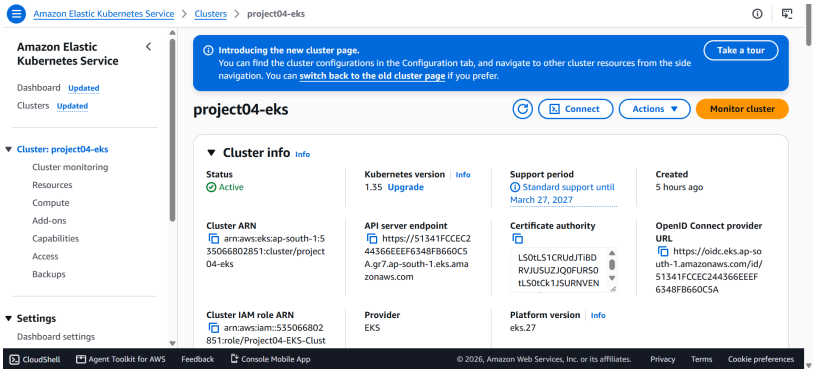


Figure 13. AWS EKS cluster console — project04-eks Active, Kubernetes 1.35.

```
sh: syntax error near unexpected token `newline'
sh-5.2$ kubectl get nodes -o wide
NAME                                         STATUS   ROLES    AGE    VERSION              INTERNAL-IP     EXTERNAL-IP     OS-IMAGE
     KERNEL-VERSION                  CONTAINER-RUNTIME
ip-172-31-31-129.ap-south-1.compute.internal   Ready    <none>   166m   v1.35.9-eks-577a51e   172.31.31.129   3.110.116.102   Amazon Linux 2023.12.202610
26   6.12.113-138.209.amzn2023.x86_64   containerd://2.3.7+unknown
sh-5.2$
```

Figure 14. kubectl get nodes output showing the worker node Ready.

9. Kubernetes resources — application and database

The application ran in the default namespace. The observed resources included blog-frontend, blog-backend, mysql-0, Services, ConfigMap blog-db-init, Secret blog-db-secret and a 5Gi gp3 persistent volume claim.

Step 9.1 — Inspect resources

Where: Host connected to EKS

```
kubectl get deployments -n default
kubectl get pods -n default -o wide
kubectl get services -n default
```

Expected result: Captured state: blog-frontend 1/1 Running, blog-backend 1/1 Running and mysql-0 1/1 Running. Services include frontend, backend and MySQL.

```
sh-5.2$ kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE    IP              NODE                                            NOMINATED NODE   READ
INESS GATES
blog-backend-57cd86cb49-27fh6       1/1     Running   0          126m   172.31.24.229   ip-172-31-31-129.ap-south-1.compute.internal   <none>           <non
e>
blog-frontend-569c556458-cs94t      1/1     Running   0          122m   172.31.16.8     ip-172-31-31-129.ap-south-1.compute.internal   <none>           <non
e>
mysql-0                             1/1     Running   0          129m   172.31.18.113   ip-172-31-31-129.ap-south-1.compute.internal   <none>           <non
e>
sh-5.2$
```

Figure 15. Kubernetes pod listing — frontend, backend and MySQL workloads running.

```
sh-5.2$ kubectl get services
NAME                    TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE
blog-backend-service    ClusterIP   10.100.89.26     <none>        5000/TCP   127m
blog-frontend-service   ClusterIP   10.100.163.140   <none>        80/TCP     127m
kubernetes              ClusterIP   10.100.0.1       <none>        443/TCP    4h34m
mysql-service           ClusterIP   None             <none>        3306/TCP   130m
sh-5.2$
```

Figure 16. Kubernetes Services — frontend and backend ClusterIP services plus MySQL headless service.

Step 9.2 — Inspect MySQL StatefulSet

Where: Host connected to EKS

```
kubectl get statefulset
kubectl describe statefulset mysql
```

Expected result: MySQL StatefulSet configuration was captured. The StatefulSet pod name was mysql-0.

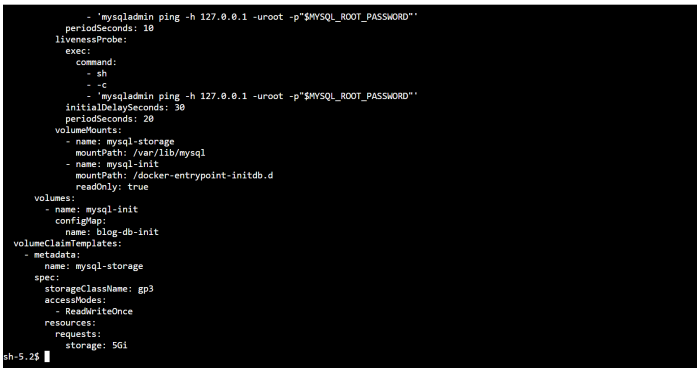


Figure 17. MySQL StatefulSet configuration evidence.

Step 9.3 — Verify persistent storage

Where: Host connected to EKS

```
kubectl get pvc
kubectl get pv
```

Expected result: Captured PVC state: Bound, 5Gi, ReadWriteOnce, gp3 storage class.

```
          storage: 5Gi
sh-5.2$ kubectl get pvc
NAME                    STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
mysql-storage-mysql-0   Bound    pvc-a1b3cfde-95ff-4398-993a-ede8bfb2896a   5Gi        RWO            gp3            <unset>                 94m
sh-5.2$
```

Figure 18. Persistent volume claim output — 5Gi Bound.

Step 9.4 — Check ConfigMap and Secret metadata

Where: Host connected to EKS

```
kubectl get configmaps -n default
kubectl get secrets -n default
```

Expected result: Captured objects: ConfigMap blog-db-init and Secret blog-db-secret (4 data entries). Never expose decoded Secret values.

```
sh-5.2$ kubectl get configmaps
NAME                DATA   AGE
blog-db-init        1      131m
kube-root-ca.crt    1      4h34m
sh-5.2$
```

Figure 19. ConfigMap listing.

```
sh-5.2$ kubectl get secrets
NAME            TYPE     DATA   AGE
blog-db-secret   Opaque   4      135m
sh-5.2$
```

Figure 20. Kubernetes Secret metadata only; secret values are intentionally not shown.

10. Deployment rollout and application validation

Jenkins build #23 updated the image references and waited for both deployments to finish rolling out.

```
kubectl set image deployment/blog-frontend frontend=535066802851.dkr.ecr.ap-south-1.amazonaws.com/blogreact-frontend:build-23 -n
default
kubectl set image deployment/blog-backend backend=535066802851.dkr.ecr.ap-south-1.amazonaws.com/blogreact-backend:build-23 -n
default

kubectl rollout status deployment/blog-frontend -n default --timeout=300s
kubectl rollout status deployment/blog-backend -n default --timeout=300s
```

Recorded Jenkins output: deployment "blog-frontend" successfully rolled out; deployment "blog-backend" successfully rolled out.

```
sh-5.2$ kubectl rollout status deployment/blog-frontend && kubectl rollout status deployment/blog-backend
deployment "blog-frontend" successfully rolled out
deployment "blog-backend" successfully rolled out
sh-5.2$
```

Figure 21. Direct kubectl rollout validation — both frontend and backend successfully rolled out.

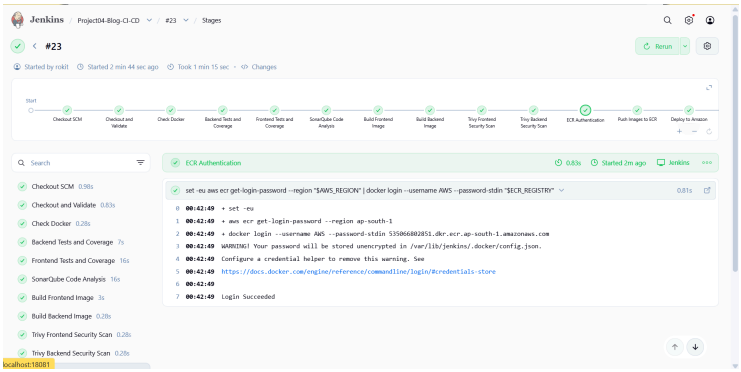


Figure 22. Jenkins build #23 stage view — ECR authentication, image push and Amazon EKS deployment stages are green.

Frontend access was tested through local port forwarding:

```
kubectl port-forward svc/blog-frontend-service 28082:80 -n default
# Browser:
http://127.0.0.1:28082
```

Backend health was tested through the service. The captured internal test pod returned {"status":"ok"} for /healthz.

```
kubectl run api-test --rm -i --restart=Never --image=curlimages/curl -- curl -sS http://blog-backend-service:5000/healthz
```

```
sh-5.2$ kubectl run api-test --rm -i --restart=Never --image=curlimages/curl -- curl -sS http://blog-backend-service:5000/healthz
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed through the command
prompt.
If you don't see a command prompt, try pressing enter.
warning: couldn't attach to pod/api-test, falling back to streaming logs: Internal error occurred: unable to upgrade connection: container api-test not f
ound in pod api-test_default
{"status":"ok"}pod "api-test" deleted from default namespace
sh-5.2$
```

Figure 23. Backend service health response from a temporary curl test pod.



Figure 24. Additional application/service validation terminal evidence.

A MySQL database inspection command was also captured. It confirms database access/listing in that session; do not overstate it as a full application CRUD or backup/restore test.

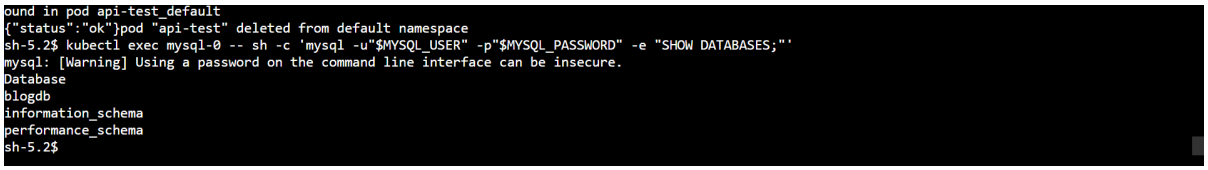


Figure 25. MySQL database inspection evidence.

11. Prometheus and Grafana monitoring

Monitoring resources were installed in namespace monitoring with the kube-prometheus-stack Helm chart family. The captured Helm release name was project04-monitoring. Prometheus was configured as the Grafana datasource.

Step 11.1 — Check monitoring pods

Where: Host connected to EKS

```
kubectl get pods -n monitoring -o wide
helm list -n monitoring
```

Expected result: Later captured state showed Grafana 3/3, Prometheus 2/2, kube-operator 1/1, kube-state-metrics 1/1 and node-exporter pods 1/1. An earlier troubleshooting capture showed Grafana 2/3 and readiness/liveness failures; the later dashboard evidence shows it became accessible.

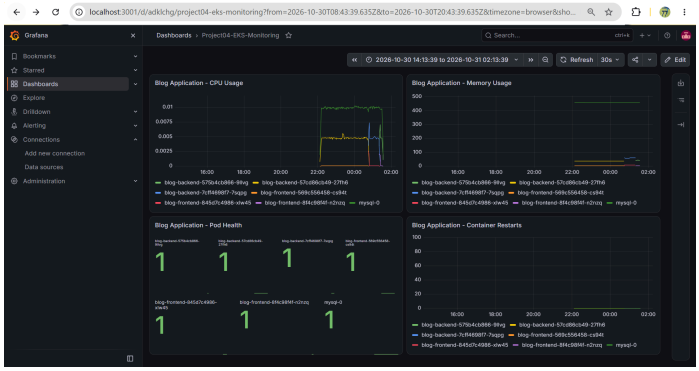


Figure 26. Grafana Project04-EKS-Monitoring dashboard.

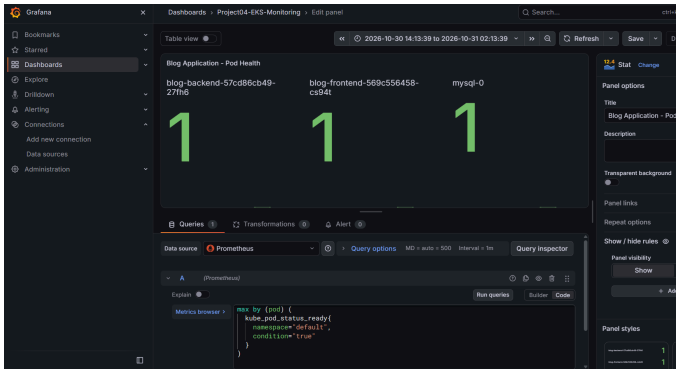


Figure 27. Grafana pod-health panel showing backend, frontend and mysql-0 readiness values.

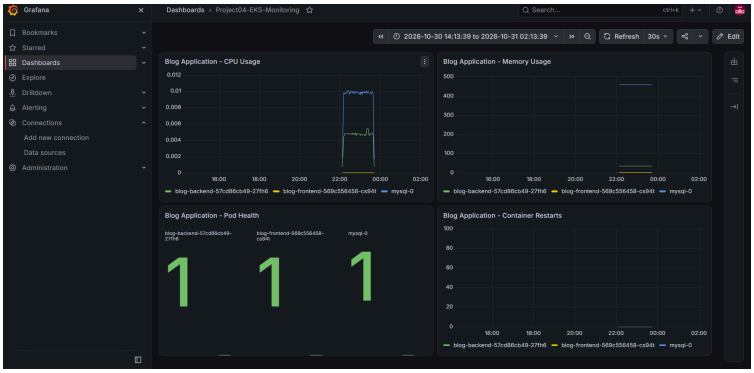


Figure 28. Grafana CPU and memory monitoring panels.

Step 11.2 — Inspect Prometheus discovery

Where: Host connected to EKS

```
kubectl get servicemonitor,podmonitor -A
kubectl get prometheus -n monitoring
```

Expected result: Captured evidence shows ServiceMonitor/Prometheus resources. The retained screenshots do not establish a complete target-by-target UP audit.

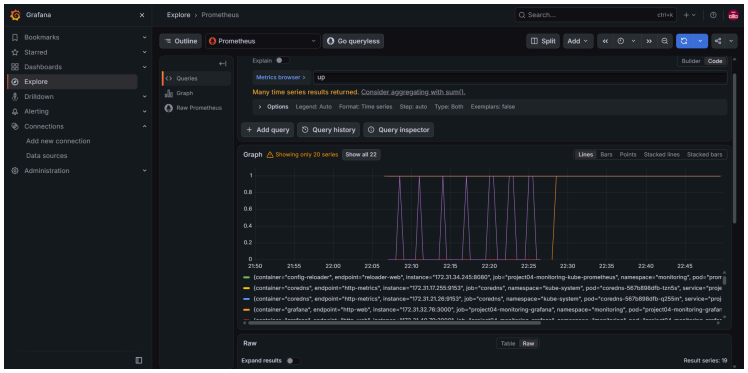


Figure 29. Prometheus Explore graph with discovered series.

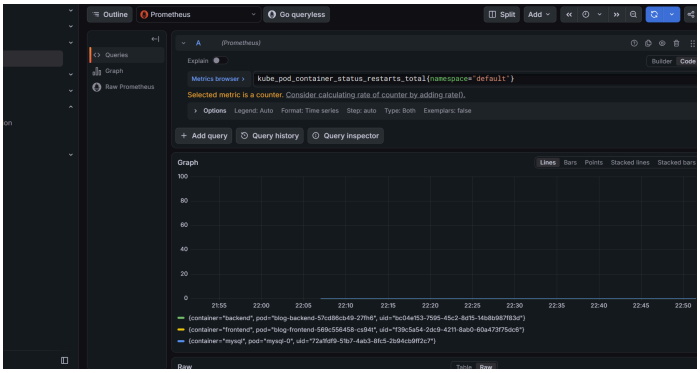


Figure 30. Prometheus query for kube_pod_container_status_restarts_total in the default namespace.

Monitoring evidence supports that the dashboards and metrics queries were accessible during the session. It does not prove that all alerts, notifications or long-term retention policies were configured and tested.

12. Troubleshooting notes from the actual run

Observed issue	What the retained evidence shows	Correct handling
Port-forward conflict	Port 18081 was already owned by a kubectl process (PID 606819).	Use a different unused local port or stop only the known port-forward process.
Grafana readiness	An earlier pod showed 2/3 readiness and probe errors.	Inspect Grafana logs/events; later dashboard and 3/3 pod evidence confirms it became accessible.
Helm release status	One troubleshooting output recorded failed/context-canceled status.	Do not describe every Helm revision as clean; use later pod/dashboard evidence as the final observed state.
Jenkins source checkout	Jenkins log says selected Git installation unavailable and used Default; no credentials specified.	Pipeline nevertheless checked out the public GitHub repository successfully.
NPM warnings	Deprecated package notices and npm audit counts appeared during installs.	These are dependency maintenance warnings; distinguish them from test failures.

Sonar coverage figures	Backend/frontend test coverage differ from Sonar New Code coverage.	Keep test-runner percentages separate from SonarQube's 93.0% New Code coverage.

Useful diagnostics:

```
kubectl get pods -A -o wide
kubectl get events -A --sort-by=.lastTimestamp
kubectl describe pod <POD_NAME> -n <NAMESPACE>
kubectl logs <POD_NAME> -n <NAMESPACE> --tail=100
kubectl logs <POD_NAME> -n <NAMESPACE> --previous --tail=100
kubectl get deployments -n default
kubectl rollout history deployment/blog-frontend -n default
```

Redact credentials, tokens, passwords, personal information and connection strings before sharing terminal logs.

13. Final evidence and acceptance matrix

Requirement	Status	What the evidence proves
Frontend application	VERIFIED	Marginal UI rendered in browser through localhost port-forward.
Backend health	VERIFIED	Temporary curl pod returned {"status":"ok"}; Jenkins tests validated /healthz and /readyz.
Backend automated tests	VERIFIED	Jest: 5 tests passed; backend coverage report retained in Jenkins log.
Frontend automated tests	VERIFIED	Vitest: 25 tests passed in 5 files.
SonarQube analysis	VERIFIED	Scanner: ANALYSIS SUCCESSFUL; dashboard Quality Gate Passed.
Sonar New Code coverage	VERIFIED	93.0% for 68 new lines; not overall application coverage.
Trivy image scans	VERIFIED	Build-23 summaries show 0 vulnerabilities for displayed frontend/backend targets.
ECR publishing	VERIFIED	Jenkins log confirms image pushes to both repositories.
EKS deployment	VERIFIED	Both deployment rollout commands succeeded.
MySQL persistent volume	PARTIAL	5Gi gp3 PVC Bound; full backup/restore and CRUD not confirmed.
Prometheus/Grafana	PARTIAL	Dashboard/queries visible; full target and alert audit not retained.

This table deliberately separates successful pipeline execution from production-readiness items that were not proven by the supplied evidence.

14. AWS cleanup and cost-control record

The deployment was intentionally removed on 31 October 2026. The EKS cluster and ECR repositories were deleted to reduce ongoing AWS charges.

```
eksctl delete cluster --name project04-eks --region ap-south-1 --wait

aws ecr delete-repository --repository-name blogreact-frontend --region ap-south-1 --force
aws ecr delete-repository --repository-name blogreact-backend --region ap-south-1 --force
```

Recorded post-cleanup checks in ap-south-1: no EKS clusters, no ECR repositories, no EBS volumes, no Elastic IPs, no ELBv2 load balancers and no NAT Gateways; three related EC2 instances were terminated.

Scope limitation: these checks cover specific resource types in ap-south-1 only. They do not prove that every AWS region/global service is empty. S3, snapshots, AWS Backup, CloudFront, Route 53, Marketplace and other regions must be checked separately.

Billing screenshots captured on 31 October show Payments due = 0 and total outstanding balance = INR 0.00. The billing page also showed estimated service usage; an estimated cost is not the same as an issued invoice or payment due.

15. Project outcome

The retained evidence supports a successful CI/CD path from GitHub checkout through backend/frontend tests, SonarQube analysis, Docker image builds, Trivy scans, ECR publishing and EKS deployment rollout. Application health checks, Kubernetes workload status, MySQL PVC binding and Grafana monitoring were also observed. The AWS deployment was subsequently deleted for cost control.

The key correction in this edition is the evidence placement: Jenkins CI stage view, Jenkins CD stage view, Jenkins SonarQube Quality Gate integration and the SonarQube dashboard are distinct screenshots with distinct explanations. Frontend and backend test stages are also documented separately. JSON is described as a detected SonarQube analysis language/profile, not as a separate runtime application component.

End of Project-04 final evidence-led report.